

Lab Companion — Datastream: PostgreSQL to BigQuery (the split lab)

Your companion for the Day 1 (Mon Oct 5, 2026) Datastream lab, which runs in two halves around lunch: **Part 1, 11:50–12:00 CT** (set up and start the stream) and **Part 2, 13:00–13:15 CT** (validate the backfill, then watch a live change land). The lab's own step-by-step instructions live in the [Google Skills classroom](#) (course 1, module 2) — this companion adds orientation, context, and the things that commonly go wrong. It follows the [Module 2 session](#) on replication and migration.

What you'll do

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database (the source).
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data — and every change you make afterwards — replicates correctly into BigQuery.

Why it matters for your migration

This lab is your project's pilot wearing a lab costume. The thing your legacy extracts do tonight — copy what changed in the database into the warehouse — this stream does continuously, with no batch window. Part 2 is deliberately scheduled after lunch so you experience the headline:

Your data changed while you ate lunch. No nightly job ran, no extract window opened, no SSIS package executed.

If you take one feeling back to work from Day 1, make it that one.

Why PostgreSQL and not SQL Server? PostgreSQL is in this lab because it's free-tier friendly — the mechanics are **identical for SQL Server, which IS a supported Datastream source**. Connection profiles, streams, backfill, CDC: everything you click today maps one-to-one onto your SQL Server migration. (Details: [Configure a source SQL Server database for Datastream](#).)

Before you start

- ☐ Your lab is launched from the Skills classroom and you can see your **lab-provided GCP project** and credentials on the lab panel. You work in that project — not any other project you may see in the console's project picker.
- ☐ Part 1 is only 10 minutes. Have the lab open and be ready to move when the block starts; the instructor will drive the setup briskly and you'll follow click-for-click.

- Nothing to install — the console does everything. The lab itself prepares the PostgreSQL source for replication; that prep is one of the objectives, so notice what it configures. That is the part your M&O team will ask about later.

Part 1 — before lunch (~10 min): get the stream running

Follow the lab's own steps. Here is the shape of what you're doing and how to think about each piece:

1. **Prepare and seed the source.** You stand up a Cloud SQL for PostgreSQL instance and import data into it. This instance plays the role of your SQL Server warehouse: the database that never stops changing.
2. **Create two connection profiles.** A connection profile is a saved, tested connection definition — the same idea as an SSIS connection manager. One points at the source (PostgreSQL), one at the destination (BigQuery).
3. **Create the stream.** You pick the two profiles, choose what to replicate, and configure how the existing data (backfill) and future changes (CDC) will flow.
4. **Start the stream — and leave it running.** The stream begins its backfill. Once it shows it's up, Part 1 is done.

Then go to lunch. **Do not pause, stop, or clean up anything** — the stream replicating while you eat *is* the lesson.

Part 2 — after lunch (~15 min): the CDC moment

1. **Validate the backfill.** In the BigQuery console, find the tables the stream created in your dataset. Compare row counts against the source and eyeball the data. Notice the extra metadata columns Datastream adds to each row — including `change_type` and `is_deleted`, which tell you what happened to the row.
2. **Make a source change.** Run the lab's UPDATE/INSERT against the Cloud SQL instance.
3. **Watch it land.** Keep an eye on the stream's status, and refresh your BigQuery view until the change appears — typically within about a minute of the source commit. No job ran; nothing was scheduled. That is what "kill the nightly extracts" means in practice.

Common gotchas

'Where are my BigQuery tables?' Destination tables appear *as the stream runs*. If you go looking before the backfill has started creating them, you'll think it's broken. Give it time, then refresh the Explorer panel.

Region mismatch. The connection profiles' region must match the Cloud SQL instance and the BigQuery dataset location. A mismatched region is the classic first-timer failure in this lab — if a connection test or a stream fails early, check regions first.

Where to watch stream status. Status lives on the **Datastream** → **Streams** page in the console. Streams take a little time to come up (Starting → Running); that startup latency is normal, not an error.

Wrong project in the picker. The console remembers projects from other sessions. Before you create anything, confirm the project picker at the top shows your lab project — otherwise your instance, profiles, or stream land somewhere you can't see them.

API-enable prompts. First use of Datastream or Cloud SQL in a fresh project can pop "API not enabled" prompts. Click **Enable** and continue; it happens once per project.

'My change hasn't shown up in BigQuery.' After-lunch changes usually land within a minute or so. If yours hasn't: refresh the table preview, then check the stream status page before assuming failure.

Stream won't start and the clock says lunch? Don't burn the lunch window debugging. Watch the after-lunch validation live — the moment lands the same on a shared screen — and re-run the lab yourself in the evening if you want the clicks.

Cost awareness (for real life, not today). Today's resources live in a time-boxed lab project. In your own environment, a running stream and a Cloud SQL instance bill while they're up — plan for them as always-on resources, not batch jobs.

If you finish early

- **Part 1:** open your stream's details page and watch the status and throughput as the backfill begins. Compare with a neighbor whose stream started at a different moment.
- **Part 2:** make a second change of a different kind — a DELETE — and find how the metadata columns (`change_type`, `is_deleted`) represent it in BigQuery.

Self-check

Part 1, before you leave for lunch:

- ☐ Both connection profiles exist — one for the PostgreSQL source, one for the BigQuery destination.
- ☐ The stream is created and its status shows it starting or running.
- ☐ You have paused, stopped, and deleted nothing; everything stays up over lunch.

Part 2:

- ☐ You found the tables Datastream created in BigQuery, and their row counts match the source.
- ☐ You can point at the metadata columns (`change_type`, `is_deleted`) on a replicated row.
- ☐ Your own UPDATE/INSERT from the source appeared in BigQuery without you running any load job.

What this replaces in your current stack

- **The nightly extract window** → an always-on stream; freshness stops depending on a schedule.
- **The SQL Agent job + SSIS extract package** → Datastream backfill + CDC; there is no package to deploy or retry.
- **Hand-rolled delta detection** (timestamp columns, triggers, checksums) → the database change log, which already records every change.

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.